

Módulo 04 - Carrinho, Pedido, Entrega e Contrato

Objetivo

Estudar a transição do catálogo para o fluxo comercial: carrinho por sessão, conversão em pedido, cálculo de entrega, snapshots e aceite de contrato.

Commits Estudados

- `7ad2e9c` - carrinho inicial de pedidos.
- `a524126` - pedidos e validação de dados do cliente.
- `b2bcfbc` - oculta snapshot interno da API de pedidos.
- `f1e8a6f` - taxa de entrega e retirada por distância.
- `ed5c440` - provider Google Routes.
- `2c50443` - integra taxa de entrega na conversão do carrinho.
- `13464c8` - contrato vigente e aceite no pedido.

Aula 1 - Carrinho Por Sessão

O commit `7ad2e9c` cria o app `pedidos` e o carrinho inicial.

Código para ler:

- `backend/pedidos/models.py`
- `backend/pedidos/services.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/views.py`
- `backend/pedidos/urls.py`
- `backend/pedidos/tests.py`

Modelos centrais:

- `Carrinho`

- `ItemCarrinho`

O carrinho pode pertencer a:

- uma `session_key` anônima;
- um usuário autenticado;
- ambos, durante a transição após login.

No service atual:

```
def garantir_session_key(request):  
    if not request.session.session_key:  
        request.session.create()  
    return request.session.session_key
```

Decisão real:

- Visitante pode montar carrinho.
- Login só é obrigatório para fechar pedido.

Testes para estudar:

- `test_carrinho_anonimo_e_criado_e_recuperado_por_sessao`
- `test_usuario_nao_acessa_item_de_carrinho_de_outra_sessao`
- `test_adicionar_item_ao_carrinho_nao_reserva_estoque`

Aula 2 - Pedido Como Snapshot Comercial

O commit `a524126` adiciona `Pedido` e `ItemPedido`, além da conversão do carrinho.

Código para ler:

- `backend/pedidos/models.py`
- `backend/pedidos/services.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/views.py`
- `backend/pedidos/tests.py`

O pedido salva snapshots:

- nome do cliente;
- telefone;

- e-mail;
- data pretendida;
- subtotal;
- itens com preço e composição daquele momento.

Ponto de domínio:

- Se o preço do brinquedo mudar depois, o pedido antigo não deve mudar.
- Se um kit mudar depois, o pedido antigo preserva a composição.

Testes para estudar:

- `test_snapshots_do_pedido_usam_dados_do_momento_da_conversao`
- `test_converte_carrinho_com_kit_festa_preservando_snapshot`
- `test_converte_carrinho_com_kit_personalizado_preservando_snapshot`

Aula 3 - Corrigir Exposição de Snapshot

O commit `b2bcfbc` remove snapshot interno da API de pedidos.

Código para ler:

- `backend/pedidos/serializers.py`
- `backend/pedidos/tests.py`

Esse é um fix de superfície pública.

O snapshot continua salvo no banco, mas deixa de ser exposto diretamente ao cliente.

Testes para estudar:

- `test_resposta_da_conversao_nao_expoe_snapshot_do_item_pedido`
- `test_snapshot_do_item_pedido_continua_salvo_no_banco_apos_conversao`

Lição:

- Persistência e resposta pública não precisam ter o mesmo formato.
- Serializer é fronteira de segurança e produto.

Aula 4 - Taxa de Entrega e Retirada Por Distância

O commit `f1e8a6f` cria o app `entregas`.

Código para ler:

- `backend/entregas/models.py`
- `backend/entregas/providers.py`
- `backend/entregas/services.py`
- `backend/entregas/views.py`
- `backend/entregas/tests.py`

Conceitos reais:

- `ConfiguracaoTaxaEntregaRetirada` guarda origem e valor por km.
- `CepProvider` interpreta CEP.
- `RotaProvider` calcula distância.
- `TaxaEntregaRetiradaService` calcula ida, volta e taxa.

No service atual:

```
distancia_total_km = quantizar_decimal(distancia_ida_km * Decimal("2"))
taxa = quantizar_decimal(distancia_total_km * valor_por_km)
```

Ponto crítico:

- Distância, valor por km e taxa não são aceitos do frontend.
- O backend calcula e retorna o valor.

Teste para estudar:

- `test_frontend_nao_consegue_forjar_distancia_taxa_ou_valor_por_km`

Aula 5 - Provider Google Routes

O commit `ed5c440` troca o provider fake inicial por integração com Google Routes.

Código para ler:

- `backend/entregas/providers.py`
- `backend/entregas/tests.py`
- `backend/setup/settings.py`

Pontos reais:

- `GOOGLE_ROUTES_API_KEY` vem de settings.

- O provider falha de forma segura sem chave.
- Timeout, HTTP inválido e resposta sem rota viram erro controlado.

Testes para estudar:

- `test_provider_falha_de_forma_segura_sem_chave_configurada`
- `test_provider_falha_de_forma_segura_em_timeout`
- `test_provider_falha_se_api_retornar_resposta_sem_rota`

Aula 6 - Entrega Integrada ao Pedido

O commit `2c50443` integra a taxa de entrega na conversão do carrinho.

Código para ler:

- `backend/pedidos/models.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/services.py`
- `backend/pedidos/tests.py`

Campos de snapshot introduzidos ou usados:

- `endereco_entrega_snapshot`
- `distancia_ida_km_snapshot`
- `distancia_total_km_snapshot`
- `valor_por_km_snapshot`
- `taxa_entrega_retirada_snapshot`
- `total_estimado_snapshot`

Serializer bloqueia campos forjados:

```
campos_proibidos = {  
    "distancia_km",  
    "taxa",  
    "frete",  
    "total_estimado_snapshot",  
}
```

Testes para estudar:

- `test_pedido_salva_taxa_entrega_retirada_snapshot`

- `test_pedido_salva_total_estimado_snapshot_corretamente`
- `test_frontend_nao_consegue_forjar_taxa_distancia_valor_por_km_ou_total`
- `test_falha_no_calculo_da_taxa_nao_converte_o_carrinho`

Aula 7 - Contrato e Aceite

O commit `13464c8` adiciona `Contrato` e `AceiteContrato`.

Código para ler:

- `backend/pedidos/models.py`
- `backend/pedidos/serializers.py`
- `backend/pedidos/services.py`
- `backend/pedidos/views.py`
- `backend/pedidos/urls.py`
- `backend/pedidos/tests.py`

Regras reais:

- existe um contrato ativo;
- cliente acessa contrato do próprio pedido;
- aceite registra versão, texto, cliente, IP, user-agent e data;
- frontend não pode forjar dados de auditoria;
- contrato com aceite não pode ter texto ou versão alterados.

Trecho conceitual:

```
if campos_forjados:
    raise serializers.ValidationError({
        "detail": "Dados de auditoria do aceite sao registrados pelo backend."
    })
```

Testes para estudar:

- `test_aceite_salva_snapshots_e_auditoria`
- `test_frontend_nao_consegue_forjar_dados_de_auditoria_do_aceite`
- `test_contrato_alterado_depois_nao_altera_snapshot_do_aceite`
- `test_contrato_com_aceite_nao_pode_alterar_texto_ou_versao`

Revisão do Módulo

Você deve conseguir responder:

- Por que carrinho não reserva estoque?
- Por que pedido precisa salvar snapshot?
- Onde o backend bloqueia preço, taxa e total enviados pelo frontend?
- Qual diferença entre contrato vigente e aceite do pedido?
- Que testes impedem forja de dados de auditoria?

Exercício Prático

Abra `git show 2c50443` e identifique o caminho completo:

1. serializer recebe dados do checkout;
2. service calcula taxa;
3. pedido salva snapshots;
4. testes impedem campos financeiros forjados.